

Assignment 4: Data Wrangling (Spring 2026)

Leo Zhang

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Wrangling. Do not use any AI tools in completing this assignment. LZ

Directions

1. Rename this file `<FirstLast>_A04_DataWrangling.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. **Ensure that code in code chunks is tidy and does not extend off the page in the PDF.**
7. **Push your completed RMD to your GitHub account**

Set up your session

- 1a. Load the `tidyverse` and `here` packages into your session.
 - 1b. Check your working directory.
 - 1c. Read in all four raw data files associated with the EPA Air dataset, being sure to set string columns to be read in as factors. See the README file for the EPA air datasets for more information (especially if you have not worked with air quality data previously).
2. Add the appropriate code to reveal the dimensions of the four datasets.

```
#1a
knitr::opts_chunk$set(tidy = TRUE, tidy.opts = list(width.cutoff = 20))
library(here)
library(tidyverse)

#1b
getwd()
```

```
## [1] "C:/Users/24169/Desktop/e872/872_spring26"
```

```
#1c
ozone_old <- read.csv(here('./Data/Raw/EPAair_03_NC2018_raw.csv'), stringsAsFactors = TRUE)
ozone_new <- read.csv(here('./Data/Raw/EPAair_03_NC2019_raw.csv'), stringsAsFactors = TRUE)
particle_old <- read.csv(here('./Data/Raw/EPAair_PM25_NC2018_raw.csv'), stringsAsFactors = TRUE)
```

```
particle_new <- read.csv(here('./Data/Raw/EPAair_PM25_NC2019_raw.csv'), stringsAsFactors = TRUE)
#2
dim(ozone_old)
```

```
## [1] 9737    20
```

```
dim(ozone_new)
```

```
## [1] 10592    20
```

```
dim(particle_old)
```

```
## [1] 8983     20
```

```
dim(particle_new)
```

```
## [1] 8581     20
```

TIP: All four datasets should have the same number of columns but unique record counts (rows). Do your datasets follow this pattern?

Answer: Yes, all four dataframes have 20 columns.

Wrangle individual datasets to create processed files.

3. Change any date columns to be date objects.
4. Create new dataframes with just the following columns: 'Date', 'DAILY_AQI_VALUE', 'Site.Name', 'AQS_PARAMETER_DESC', 'COUNTY', 'SITE_LATITUDE', 'SITE_LONGITUDE'
5. For the PM2.5 datasets, fill all cells in AQS_PARAMETER_DESC with "PM2.5" (all cell values in this column should be identical).
6. Save all four processed datasets in the Processed folder. Use the same file names as the raw files but replace "raw" with "processed".

```
# 3
ozone_old <- ozone_old |>
  mutate(Date = mdy(Date))
class(ozone_old$Date)
```

```
## [1] "Date"
```

```
ozone_new <- ozone_new |>
  mutate(Date = mdy(Date))
class(ozone_new$Date)
```

```
## [1] "Date"
```

```
particle_old <- particle_old |>
  mutate(Date = mdy(Date))
class(particle_old$Date)
```

```
## [1] "Date"
```

```
particle_new <- particle_new |>
  mutate(Date = mdy(Date))
class(particle_new$Date)
```

```
## [1] "Date"
```

```
# 4
old_o <- ozone_old |>
  select(Date, DAILY_AQI_VALUE,
    Site.Name, AQS_PARAMETER_DESC,
    COUNTY, SITE_LATITUDE,
    SITE_LONGITUDE)
new_o <- ozone_new |>
  select(Date, DAILY_AQI_VALUE,
    Site.Name, AQS_PARAMETER_DESC,
    COUNTY, SITE_LATITUDE,
    SITE_LONGITUDE)
old_p <- particle_old |>
  select(Date, DAILY_AQI_VALUE,
    Site.Name, AQS_PARAMETER_DESC,
    COUNTY, SITE_LATITUDE,
    SITE_LONGITUDE)
new_p <- particle_new |>
  select(Date, DAILY_AQI_VALUE,
    Site.Name, AQS_PARAMETER_DESC,
    COUNTY, SITE_LATITUDE,
    SITE_LONGITUDE)

# 5
old_p <- old_p |>
  mutate(AQS_PARAMETER_DESC = "PM2.5")
new_p <- new_p |>
  mutate(AQS_PARAMETER_DESC = "PM2.5")

# 6
write.csv(old_o, row.names = FALSE,
  file = here("./Data/Processed/EPAair_03_NC2018_processed.csv"))
write.csv(new_o, row.names = FALSE,
  file = here("./Data/Processed/EPAair_03_NC2019_processed.csv"))
write.csv(old_p, row.names = FALSE,
  file = here("./Data/Processed/EPAair_PM25_NC2018_processed.csv"))
write.csv(new_p, row.names = FALSE,
  file = here("./Data/Processed/EPAair_PM25_NC2019_processed.csv"))
```

Combine datasets

- Combine the four datasets with `rbind`. Make sure your column names are identical prior to running this code.

8. Use code to display the dimensions of the combined dataset.
9. Wrangle your new dataset with a pipe function (`%>%`) so that it fills the following conditions:
 - Include only sites that the four data frames have in common: “Linville Falls”, “Durham Armory”, “Leggett”, “Hattie Avenue”, “Clemmons Middle”, “Mendenhall School”, “Frying Pan Mountain”, “West Johnston Co.”, “Garinger High School”, “Castle Hayne”, “Pitt Agri. Center”, “Bryson City”, “Millbrook School”
 - Some sites have multiple measurements per day. Use the split-apply-combine strategy to generate daily means: group by date, site name, AQS parameter, and county. Take the mean of the AQI value, latitude, and longitude.
 - Add new columns for “Month” and “Year” by parsing your “Date” column (hint: `lubridate` package)
 - Hint: the dimensions of this dataset should be 14,752 x 9.
10. Spread your datasets such that AQI values for ozone and PM2.5 are in separate columns. Each location on a specific date should now occupy only one row.
11. Call up the dimensions of your new tidy dataset.
12. Save your processed dataset with the following file name: “EPAair_O3_PM25_NC1819_Processed.csv”

```
# 7
air_overview <- rbind(old_o,
  old_p, new_o, new_p)
# 8
str(air_overview)
```

```
## 'data.frame': 37893 obs. of 7 variables:
## $ Date : Date, format: "2018-03-01" "2018-03-02" ...
## $ DAILY_AQI_VALUE : int 40 43 44 45 44 28 33 41 45 40 ...
## $ Site.Name : Factor w/ 51 levels "", "Beaufort",...: 35 35 35 35 35 35 35 35 35 35 ...
## $ AQS_PARAMETER_DESC: Factor w/ 2 levels "Ozone", "PM2.5": 1 1 1 1 1 1 1 1 1 1 ...
## $ COUNTY : Factor w/ 37 levels "Alexander", "Avery",...: 1 1 1 1 1 1 1 1 1 1 ...
## $ SITE_LATITUDE : num 35.9 35.9 35.9 35.9 35.9 ...
## $ SITE_LONGITUDE : num -81.2 -81.2 -81.2 -81.2 -81.2 ...
```

```
# 9
air_overview <- air_overview %>%
  filter(Site.Name ==
    "Linville Falls" |
    Site.Name ==
    "Durham Armory" |
    Site.Name ==
    "Leggett" |
    Site.Name ==
    "Hattie Avenue" |
    Site.Name ==
    "Clemmons Middle" |
    Site.Name ==
    "Mendenhall School" |
    Site.Name ==
    "Frying Pan Mountain" |
```

```

    Site.Name ==
      "West Johnston Co." |
    Site.Name ==
      "Garinger High School" |
    Site.Name ==
      "Castle Hayne" |
    Site.Name ==
      "Pitt Agri. Center" |
    Site.Name ==
      "Bryson City" |
    Site.Name ==
      "Millbrook School") %>%
group_by(Date, Site.Name,
  AQS_PARAMETER_DESC,
  COUNTY) %>%
summarise(AQI_mean = mean(DAILY_AQI_VALUE),
  Latitude_mean = mean(SITE_LATITUDE),
  Longitude_mean = mean(SITE_LONGITUDE)) %>%
mutate(Month = month(Date),
  Year = year(Date))
# 10
air_clean <- air_overview |>
  pivot_wider(names_from = AQS_PARAMETER_DESC,
    values_from = AQI_mean)
# 11
str(air_clean)

```

```

## gropd_df [8,976 x 9] (S3: grouped_df/tbl_df/tbl/data.frame)
## $ Date      : Date[1:8976], format: "2018-01-01" "2018-01-01" ...
## $ Site.Name  : Factor w/ 51 levels "", "Beaufort",...: 6 10 12 16 18 19 23 28 32 40 ...
## $ COUNTY     : Factor w/ 37 levels "Alexander","Avery",...: 29 24 10 8 22 10 9 31 26 16 ...
## $ Latitude_mean : num [1:8976] 35.4 34.4 36 36 35.2 ...
## $ Longitude_mean: num [1:8976] -83.4 -77.8 -80.3 -78.9 -80.8 ...
## $ Month      : num [1:8976] 1 1 1 1 1 1 1 1 1 1 ...
## $ Year       : num [1:8976] 2018 2018 2018 2018 2018 ...
## $ PM2.5      : num [1:8976] 35 13 24 31 20 22 14 28 15 24 ...
## $ Ozone      : num [1:8976] NA NA NA NA 32 NA NA 34 NA NA ...
## - attr(*, "groups")= tibble [8,976 x 3] (S3: tbl_df/tbl/data.frame)
## ..$ Date      : Date[1:8976], format: "2018-01-01" "2018-01-01" ...
## ..$ Site.Name: Factor w/ 51 levels "", "Beaufort",...: 6 10 12 16 18 19 23 28 32 40 ...
## ..$ .rows     : list<int> [1:8976]
## .. ..$ : int 1
## .. ..$ : int 2
## .. ..$ : int 3
## .. ..$ : int 4
## .. ..$ : int 5
## .. ..$ : int 6
## .. ..$ : int 7
## .. ..$ : int 8
## .. ..$ : int 9
## .. ..$ : int 10
## .. ..$ : int 11
## .. ..$ : int 12

```

```
## .. ..$ : int 13
## .. ..$ : int 14
## .. ..$ : int 15
## .. ..$ : int 16
## .. ..$ : int 17
## .. ..$ : int 18
## .. ..$ : int 19
## .. ..$ : int 20
## .. ..$ : int 21
## .. ..$ : int 22
## .. ..$ : int 23
## .. ..$ : int 24
## .. ..$ : int 25
## .. ..$ : int 26
## .. ..$ : int 27
## .. ..$ : int 28
## .. ..$ : int 29
## .. ..$ : int 30
## .. ..$ : int 31
## .. ..$ : int 32
## .. ..$ : int 33
## .. ..$ : int 34
## .. ..$ : int 35
## .. ..$ : int 36
## .. ..$ : int 37
## .. ..$ : int 38
## .. ..$ : int 39
## .. ..$ : int 40
## .. ..$ : int 41
## .. ..$ : int 42
## .. ..$ : int 43
## .. ..$ : int 44
## .. ..$ : int 45
## .. ..$ : int 46
## .. ..$ : int 47
## .. ..$ : int 48
## .. ..$ : int 49
## .. ..$ : int 50
## .. ..$ : int 51
## .. ..$ : int 52
## .. ..$ : int 53
## .. ..$ : int 54
## .. ..$ : int 55
## .. ..$ : int 56
## .. ..$ : int 57
## .. ..$ : int 58
## .. ..$ : int 59
## .. ..$ : int 60
## .. ..$ : int 61
## .. ..$ : int 62
## .. ..$ : int 63
## .. ..$ : int 64
## .. ..$ : int 65
## .. ..$ : int 66
```

```
## .. ..$ : int 67
## .. ..$ : int 68
## .. ..$ : int 69
## .. ..$ : int 70
## .. ..$ : int 71
## .. ..$ : int 72
## .. ..$ : int 73
## .. ..$ : int 74
## .. ..$ : int 75
## .. ..$ : int 76
## .. ..$ : int 77
## .. ..$ : int 78
## .. ..$ : int 79
## .. ..$ : int 80
## .. ..$ : int 81
## .. ..$ : int 82
## .. ..$ : int 83
## .. ..$ : int 84
## .. ..$ : int 85
## .. ..$ : int 86
## .. ..$ : int 87
## .. ..$ : int 88
## .. ..$ : int 89
## .. ..$ : int 90
## .. ..$ : int 91
## .. ..$ : int 92
## .. ..$ : int 93
## .. ..$ : int 94
## .. ..$ : int 95
## .. ..$ : int 96
## .. ..$ : int 97
## .. ..$ : int 98
## .. ..$ : int 99
## .. .. [list output truncated]
## .. ..@ ptype: int(0)
## ..- attr(*, ".drop")= logi TRUE
```

```
# 12
write.csv(air_clean,
  row.names = FALSE,
  file = here("./Data/Processed/EPAair_03_PM25_NC1819_Processed.csv"))
```

Generate summary tables

13. Use the split-apply-combine strategy to generate a summary data frame. Data should be split into groups by site, month, and year. Then compute the mean AQI values for ozone and PM2.5 for each group. Finally, add a pipe to remove instances where the mean **ozone** values are not available (use the function `drop_na` in your pipe). It's ok to have missing mean PM2.5 values in this result.
14. Call up the dimensions of the summary dataset.

```
# 13
breath_statistics <- air_clean %>%
```

```

group_by(Site.Name,
         Month, Year) %>%
drop_na(Ozone) %>%
summarise(Ozone_mean = mean(Ozone),
          PM_mean = mean(PM2.5))
# 14
glimpse(breath_statistics)

```

```

## Rows: 239
## Columns: 5
## Groups: Site.Name, Month [127]
## $ Site.Name <fct> Bryson City, Bryson City, Bryson City, Bryson City, Bryson ~
## $ Month <dbl> 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7, 8, 8, 9, 9, 10, 10, 2, 3, ~
## $ Year <dbl> 2019, 2018, 2019, 2018, 2019, 2018, 2019, 2018, 2019, 2018, ~
## $ Ozone_mean <dbl> 32.40000, 41.58065, 42.51613, 44.53333, 45.40000, 35.93333, ~
## $ PM_mean <dbl> 26.66667, 34.74194, NA, 28.16667, 26.73333, NA, NA, NA, NA, ~

```

15. Why did we use the function `drop_na` rather than `na.omit`? Hint: replace `drop_na` with `na.omit` in part 13 and observe what happens with the dimensions of the summary data frame.

Answer: If I use `na.omit()`, I will see fewer rows. Rows with absent 'PM2.5' values are also deleted.

16. Stage, commit, and push your Assignment to your GitHub account. Provide a link to your repository below.

GitHub repository URL: https://github.com/leozhangzzgx/872_spring26